# Drawing Deep-Learning Architectures

A survey of diagram styles, conventions and tools — and what they imply for Tensorgami

October 2026

**Abstract.** Architecture diagrams are the most common figure in deep-learning papers, yet no notation standard exists: most figures are drawn by hand shortly before publication, and automatic tools mostly target debugging rather than communication. This report surveys how deep-learning models are drawn. It covers (i) a taxonomy of nine static diagram styles plus interactive explainers, (ii) the canonical depictions of the main architecture families (MLPs, CNNs, residual and U-shaped networks, RNN/LSTM/GRU, ConvLSTM, autoencoders and VAEs, GANs, Transformers), (iii) the tool landscape, from operator-graph viewers (TensorBoard, Netron, Model Explorer) to publication-oriented generators (NN-SVG, PlotNeuralNet, Net2Vis), and (iv) the available empirical evidence on what makes such diagrams effective. It closes with ten design requirements for Tensorgami, a locally installed Python tool for drawing the architecture and information flow of ML models.

## 1 Introduction

Diagrams appear in almost every paper that introduces a new neural system [1]. Readers use them for a holistic overview of the system, and some use them to decide whether a paper is relevant at all, before reading the text [2]. Outside of publishing, architecture drawings help developers **understand and debug** a model (is the skip connection where I think it is? which tensor shape enters the decoder?) and help **teach** the mechanics of a cell or block [3]. These three purposes (communication, comprehension, teaching) call for different levels of detail and fidelity. That difference is a recurring theme in this survey.

Despite this ubiquity, Marshall et al. note that there are currently no established, evidence-based conventions for presenting neural-network diagrams [2]. The Net2Vis authors observed that most figures are hand-crafted shortly before publication, which produces inconsistent conventions, wasted effort and occasional errors. They also observed that the automatic tools of the time were designed for debugging networks rather than for producing publication figures [4].

**Scope and method.** We reviewed (i) the architecture papers whose figures established today's visual conventions, (ii) the drawing tools for these models and the visual grammars they implement, and (iii) the empirical and HCI work on diagram effectiveness. We focus on **static** depictions of architecture and information flow. Visualisations of learned content (saliency maps, feature visualisation, activation atlases) are out of scope, as is most of the training-dynamics tooling. Figures 1–3 are schematic re-drawings made for this report, not reproductions of the original artwork.

## 2 A taxonomy of diagram styles

Across papers and tools we identify nine recurring static styles (Figure 1, Table 1). They differ along three axes: *what a glyph stands for* (a neuron, a tensor, an operator, an axis), *how much of the computation is shown* (from system level down to individual gates), and *which dimension the layout encodes* (data flow, time, resolution).

**S1 Neuron graph** (Figure 1 a). Units are circles and weights are lines. This is the textbook picture of an MLP and the basis of the Asimov Institute's *Neural Network Zoo*, which compares dozens of architectures by colour-coding cell types [5]. NN-SVG generates it parametrically [6]. It shows connectivity patterns intuitively (dense, sparse, recurrent, stochastic units), but it stops working beyond a few dozen units. Its explicit unit counts also suggest a precision that is rarely intended, which Marshall et al. warn against [2].

**S2 Feature-map planes** (Figure 1 b). Starting with LeNet-5 [7], convolutional layers are drawn as stacks of 2-D maps whose size shows spatial resolution and whose count shows channels. The style suits small CNNs and teaching, but becomes unreadable once a layer has hundreds of channels.

**S3 Tensor volumes** (Figure 1 c). The AlexNet figure [8] made the 2.5-D cuboid the emblem of CNNs. Height and depth stand for spatial size and width for channel count, and small inset cuboids mark kernels. PlotNeuralNet (LaTeX/TikZ) [9], NN-SVG's "AlexNet" mode [6] and visualkeras' layered view [10] produce this style. Net2Vis flattens it to 2-D glyphs whose height encodes spatial resolution and whose width encodes the number of features [4]. Marshall et al. cite the 3-D format as **the** available convention for CNNs [2]. Its weaknesses are perspective, dimensions that are rarely to scale (they are usually log-scaled or clipped), and poor support for branching topologies.

**S4 Layer-block flowchart** (Figure 1 d). Each layer or block is a (rounded) rectangle and arrows carry tensors. This is the dominant style in modern papers: residual and Inception blocks [11], [12] and the Transformer [13] are drawn this way. The Transformer figure shows the style's typical devices: fill colour by sub-layer type, a frame with an "N×" label instead of drawing repeats, and residual arrows that bypass blocks. Keras' `plot_model` [14] and torchview [15] generate it via Graphviz [16]. It scales through hierarchy and handles branches and merges, but it loses tensor sizes unless edges are annotated.

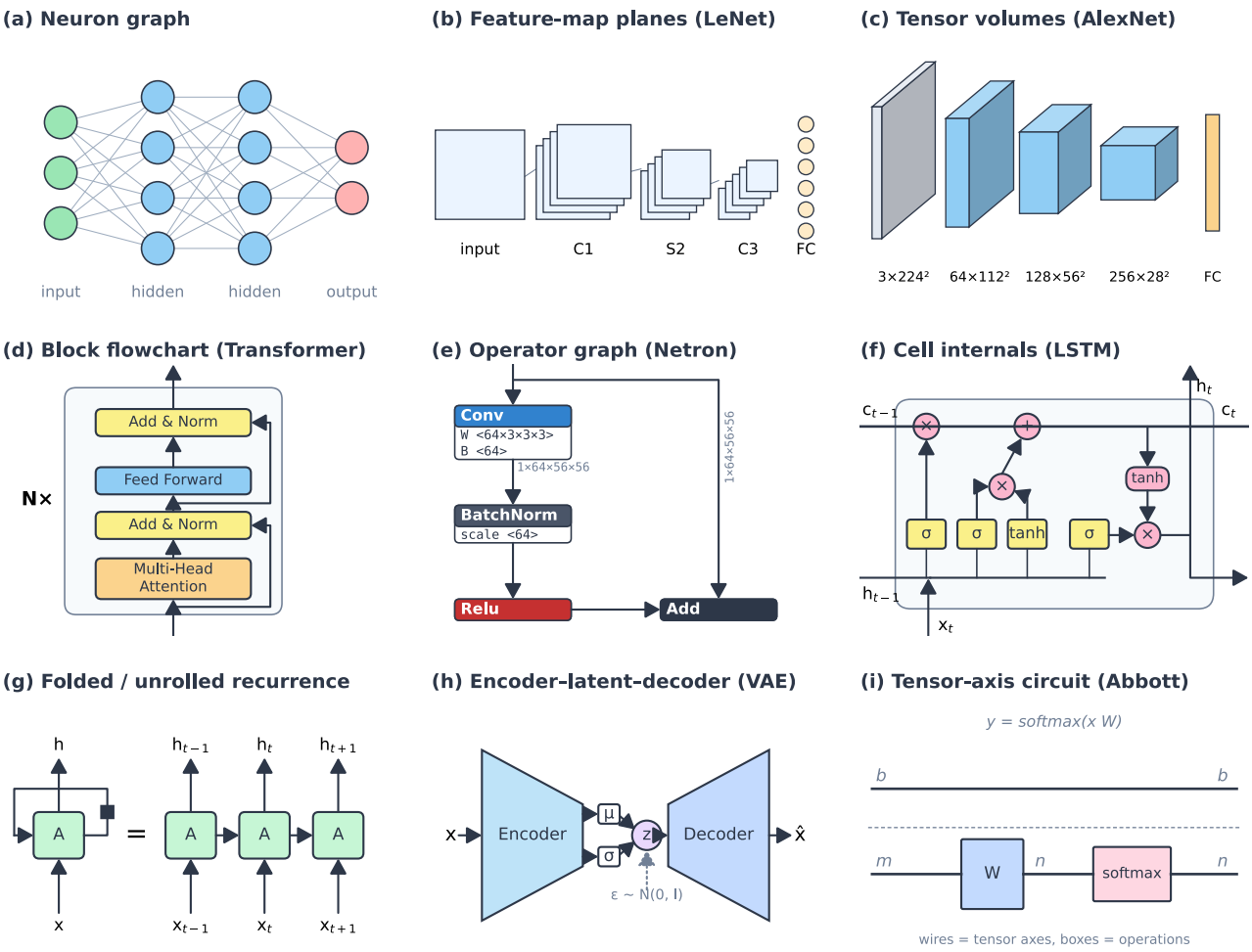



Figure 1: Nine static styles for drawing neural networks (schematic re-drawings). (a) neuron graph; (b) LeNet-style feature-map planes; (c) AlexNet/PlotNeuralNet tensor volumes; (d) layer-block flowchart with an "N×" repeat; (e) operator dataflow graph as in Netron; (f) gate-level LSTM cell after Olah; (g) folded vs. unrolled recurrence with a delay marker; (h) encoder–latent–decoder hourglass; (i) tensor-axis circuit diagram after Abbott.

**S5 Operator dataflow graph** (Figure 1 e). Every primitive operation becomes a node and every tensor an edge, often with attribute rows and shape labels. Examples are the TensorBoard graph [17], Netron [18], PyTorchViz's autograd graph [19], TorchLens [20] and Google's Model Explorer [21]. These graphs are exact and fully automatic, but real models produce thousands of nodes, so every tool in this class must collapse hierarchy and hide clutter to stay legible (§5).

**S6 Cell internals** (Figure 1 f). Gated recurrent cells are explained at gate level. Olah's LSTM diagrams [22] established the vocabulary: the cell state is a horizontal "conveyor belt", learned layers ($\sigma$, tanh) are boxes and point-wise operations ($\times$, $+$) are circles. The same grammar is reused for GRUs [23] and for many attention variants. This style explains the mathematics of a cell, but it applies only at the lowest level of detail.

**S7 Folded vs. unrolled recurrence** (Figure 1 g). A recurrent network is drawn either as a circuit with a self-loop (a black square marks the one-step delay) or unfolded into a chain over time [24]. Unrolling makes time an explicit layout axis, usually horizontal, with network depth stacked vertically. Sequence-to-sequence models [25] and attention-based decoders [26] are almost always drawn unrolled.

**S8 Encoder–latent–decoder** (Figure 1 h). Trapezoids that narrow and then widen show compression into, and expansion out of, a latent code: the hourglass of autoencoders and VAEs [27]. The U-Net [28] bends the hourglass into a "U", in which *vertical position encodes resolution* and horizontal arrows carry the skip connections.

**S9 Formal tensor diagrams** (Figure 1 i). Neural circuit diagrams [29] adapt string or tensor-network diagrams: wires are tensor axes, boxes are operations, and dashed lines separate tuple elements. Broadcasting, contraction and copying become visible, and the drawings are precise enough to derive an implementation. The cost is a learning curve and density that general audiences find hard to read.

**Interactive explainers** complete the picture: CNN Explainer [30], Bycroft's 3-D LLM visualisation [31] and Treescope's foldable model trees [32]. They use hover, animation and expand/collapse, which print cannot provide, but they show that *progressive disclosure* of detail is the key to scale.

| Style | A glyph is… | Encodes | Scales to | Typical use |
|---|---|---|---|---|
| S1 Neuron graph | a unit; edges = weights | connectivity pattern | ≈ 50 units | teaching, small MLPs, taxonomies |
| S2 Planes | a feature map | resolution, #channels | ≈ 6 layers | small CNNs, teaching |
| S3 Volumes | a tensor (cuboid) | C × H × W, kernel size | ≈ 20 layers | CNN papers |
| S4 Block flowchart | a layer / block | op type (colour), topology | 100s via nesting | most papers; generic DL |
| S5 Operator graph | a primitive op | exact topology, shapes, params | $10^4$+ with collapse | debugging, deployment |
| S6 Cell internals | a gate / point-wise op | equations of one cell | one cell | LSTM/GRU/attention explanation |
| S7 Unrolled time | a cell at step $t$ | time × depth | ≈ 5 steps shown | RNN, seq2seq, ConvLSTM |
| S8 Hourglass / U | an encoder / decoder stage | compression, resolution levels | ≈ 5 levels | AE, VAE, U-Net, diffusion |
| S9 Tensor circuit | a tensor axis (wire) | contractions, broadcasting | one block | precise specification |

Table 1: Comparison of the static diagram styles. "Scales to" gives the approximate size at which the style stays legible on one page without aggregation.

A cross-cutting observation is the **duality of node semantics**. In S4, S5 and S6 nodes are *operators* and edges carry tensors. In S2, S3 and the U-Net figure, nodes are *tensors* and the operations sit on the arrows; the U-Net legend, for example, assigns one arrow colour per operation type [28]. Both conventions are established, so a drawing tool should be able to render either view from the same model description.

## 3 Architecture families and their canonical depictions

Each family has settled on a dominant style and a set of elements that readers expect to see (Figure 2, Table 2). Marshall et al. recommend meeting those expectations, and saying explicitly when an expected element such as pooling is absent [2].

**General DL and MLPs.** Small MLPs are drawn as neuron graphs (S1). Anything larger is drawn as a chain of blocks labelled with widths (e.g. "Linear 784⟶256, ReLU"). Activations are usually merged into the label of the preceding layer rather than drawn as separate nodes.

**CNNs.** LeNet and AlexNet established planes and volumes (S2, S3). VGG-era papers moved to tables and flat block chains, and ResNet and Inception made the block flowchart (S4) standard because their key ideas are topological: the identity shortcut merging at "⊕" (Figure 2 e) and parallel branches joined by concatenation [11], [12]. Readers expect to see kernel size, channel count, stride or pooling, and the type of each merge (sum vs. concatenation). An unlabelled merge node is a common source of ambiguity.

**Encoder–decoder and U-Net.** U-Net's figure [28] combines tensor-centric glyphs (feature-map bars labelled with channels and resolution) with operation-coded arrows (convolution, copy-and-crop, max-pool, up-convolution). It is the archetype for segmentation and for the denoising networks of diffusion models [33].

**RNNs, seq2seq and attention.** Recurrent models are drawn unrolled (S7), with time on the horizontal axis and stacked layers on the vertical axis. Bidirectional layers are drawn as two rows with opposite arrows. Seq2seq figures [25] separate the encoder and the decoder chains. Attention [26] adds fan-in edges from all encoder states into a context vector, and the edges are sometimes weighted by attention strength (Figure 2 f).

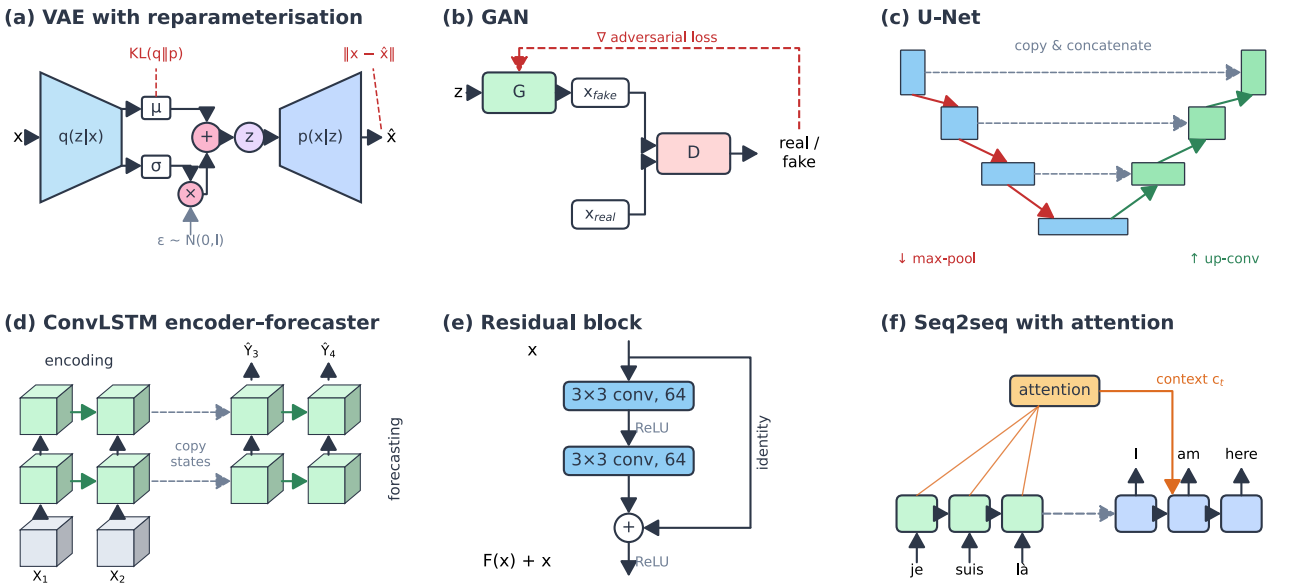



Figure 2: Canonical depictions of architecture families (schematic re-drawings). (a) VAE with explicit reparameterisation $z = \mu + \sigma \odot \varepsilon$ and loss terms drawn dashed; (b) GAN with the adversarial training loop; (c) U-Net, where vertical position encodes resolution; (d) ConvLSTM encoder–forecaster with 3-D states unrolled in time; (e) residual block with identity shortcut; (f) unrolled sequence-to-sequence model with attention.

| Family | Dominant style | Elements readers expect | Frequent pitfalls |
|---|---|---|---|
| MLP | S1 / S4 | layer widths, activation | misleading unit counts |
| CNN | S3 / S4 / S2 | kernel, channels, resolution, pooling/stride | volumes not to scale; missing pooling |
| ResNet, Inception | S4 | shortcut, merge type ($\oplus$ vs. concat), repeats | unlabelled merges |
| U-Net, AE | S8 | resolution levels, skip copies | one arrow style for many ops |
| RNN, seq2seq | S7 | time axis, state passing, direction | folded loop ambiguity |
| LSTM, GRU | S6 (+ S4 macro) | gates, cell state, point-wise ops | ad-hoc symbols |
| ConvLSTM | S7 × S3 | 3-D states, layer stack, state copy | visual overload |
| VAE | S8 + stochastic node | $\mu$, $\sigma$, $z$, $\varepsilon$, loss terms | sampling left implicit |
| GAN | S4 loop | real/fake sources, adversarial loss | training vs. inference mixed |
| Transformer | S4 + zooms | N× stacks, residual + norm, attention detail | no tensor shapes |

Table 2: Architecture families, their dominant styles (cf. Table 1) and the elements a reader expects to find.

**LSTM and GRU.** At the macro level an LSTM is a single recurrent block annotated with its hidden size. Explaining the cell requires the gate-level view (S6) [22], [23], [34], preferably next to its equations. A stable symbol set has emerged: $\sigma$ gates as boxes, point-wise $\times$ and $+$ as circles, and the cell-state line on top.

**ConvLSTM.** Shi et al. [35] replace the matrix products of the LSTM with convolutions, so inputs, hidden states and cell states become 3-D tensors. Their figures therefore combine *tensor volumes* (S3) with *unrolled time* (S7). In the encoding–forecasting network, stacked ConvLSTM layers encode the input sequence, the forecaster's initial states are copied from the encoder's last states, and the forecaster's states are concatenated and passed through a 1×1 convolution to produce the predictions (Figure 2 d). A faithful diagram must therefore show three axes: time, layer depth and the shape of each state.

**Autoencoders and VAEs.** The hourglass (S8) is universal. VAE figures add the stochastic layer: the encoder outputs $\mu$ and $\sigma$, a noise source $\varepsilon \sim \mathcal{N}(0, I)$ enters from outside, and $z = \mu + \sigma \odot \varepsilon$ is formed by the reparameterisation trick [27]. The loss terms (reconstruction and KL) are drawn as training-only annotations (Figure 2 a). The sampling node is the element most often left implicit, and drawing it explicitly removes the most common misunderstanding of the model.

**GANs.** Two networks are drawn side by side. A real-data source and the generator both feed the discriminator, and a dashed loop carries the adversarial gradient back [36]. The main design problem is to separate *training-time* flow from *inference-time* flow, a distinction the VAE and diffusion models share.

**Transformers.** The original figure [13] is a model of multi-level depiction. The macro view shows encoder and decoder stacks, each a frame marked "N×" containing coloured sub-layers (attention, feed-forward, Add & Norm) with residual arrows. Separate "zoom" figures then show multi-head and scaled dot-product attention. Graph-level tools need hierarchical collapsing to show Transformers at all: Model Explorer was built because large Transformer-based models overwhelm conventional graph visualisers [21].

## 4 Tool landscape

Table 3 summarises the software that produces such diagrams. Two clusters stand out. **Automatic, engineering-oriented** tools trace a real model and draw an exact operator or module graph. The TensorBoard graph, Netron, Model Explorer, PyTorchViz, torchview, HiddenLayer and TorchLens belong here. Their output is correct and complete, but it is styled for inspection, not for papers. **Publication-oriented** tools produce attractive figures but are manual or parametric (NN-SVG, PlotNeuralNet) or restricted to mostly sequential CNNs (visualkeras, Net2Vis). In practice many paper figures are still drawn by hand in TikZ, draw.io, PowerPoint or Figma.

This leaves a gap for a tool that extracts structure automatically and renders it with publication-grade conventions, and that also handles the non-CNN families: recurrent unrolling, ConvLSTM state tensors, stochastic latent nodes and training-only flows. Most generators also depend on Graphviz or LaTeX, which complicates local installation.

## 5 What makes an architecture diagram effective

**Corpus evidence (Net2Vis).** Bäuerle et al. collected 751 architecture figures from ICCV/CVPR 2013–2019 and interviewed authors of highly cited papers [4]. They derived a visual grammar with four parts: *visual encoding* (glyph shape encodes tensor dimensions), *layout* (data flows left to right), *layer aggregation* (repeated sequences replaced by one aggregate glyph) and *automatic legend generation*. Their user study found improvements in both creating and reading the figures.

**Making dataflow graphs legible (TensorBoard).** Wongsuphasawat et al. showed that raw operator graphs become usable only after a series of graph transformations [17]. These transformations move non-critical, high-degree nodes out of the main layout, build an expandable cluster hierarchy from the code's namespaces, and detect and highlight repeated structures. The transformed graph can then be laid out with standard layered drawing [38]. Model Explorer follows the same principle, letting users descend layer by layer through an expandable hierarchy [21].

**Reader studies (Marshall et al.).** The only evidence-based guidance specific to neural-network diagrams comes from interviews, a mixed-methods experiment and a corpus analysis [2]. All interviewees said consistency matters, and 10 of 12

| Tool | Input | Style | Layout / output | Notable features / limits |
|---|---|---|---|---|
| TensorBoard Graph [17] | TF graphs | S5 | layered, interactive | namespace clustering, auxiliary-node extraction, repeat detection |
| Netron [18] | ONNX, TFLite, Keras, PyTorch, … | S5 | interactive app | broad format support; operator level only |
| Model Explorer [21] | JAX, PyTorch, TF, TFLite | S5 | hierarchical, GPU-rendered | $10^4$+ nodes, expand/collapse, overlays |
| TorchLens [20] | PyTorch | S5 | Graphviz | all ops incl. non-module calls; activations |
| torchview [15] | PyTorch | S4/S5 | Graphviz | module depth control, shapes, `meta` device |
| PyTorchViz [19] | PyTorch autograd | S5 | Graphviz | backward graph; hard to read |
| HiddenLayer [37] | PyTorch, TF, Keras | S4 | Graphviz | rule-based folding of common op patterns |
| Keras `plot_model` [14] | Keras | S4 | Graphviz | shapes on nodes; few styling options |
| visualkeras [10] | Keras | S3/S4 | raster image | layered 3-D view; mostly sequential models |
| Net2Vis [4] | Keras-style code | S3 (2-D) | SVG/PDF, web | visual grammar, aggregation, legend |
| NN-SVG [6] | web form | S1/S2/S3 | SVG | parametric, not from a model |
| PlotNeuralNet [9] | Python ⟶ TikZ | S3 | LaTeX/PDF | high quality; manual, needs LaTeX |
| Treescope [32] | JAX/PyTorch objects | tree | HTML (notebook) | foldable model structure, arrays |

Table 3: Representative tools for drawing neural-network architectures.

considered hyperparameters unnecessary in a system diagram. Papers whose diagrams followed the resulting guidelines received more citations. The twelve guidelines are: (A) use conventional graphical objects; (B) use one arrow type for information flow and reserve other arrow types for fundamentally different uses; (C) use precision with care, since drawing four circles implies "four"; (D) show the input and output of the whole system; (E) consider a single consistent example; (F) use visual encodings (proximity, alignment) meaningfully; (G) make navigation easy with labels and linear alignment; (H) do not use colour for aesthetics, only for grouping; (I) use available conventions such as the 3-D CNN format with numeric filter sizes; (J) show what readers expect, or say when it is absent; (K) be specific ("BERT", not "embedding"); (L) make the diagram usable without the text.

**Who is looking, and why.** Hohman et al.'s interrogative survey [3] separates model developers, model users and non-experts, and separates purposes such as interpretability, debugging and comparison, and teaching. Each audience needs a different level of detail. This supports generating *several* views of one model rather than a single "correct" diagram.

**Colour and accessibility.** Colour should encode categories redundantly with labels or shapes, and palettes should stay distinguishable under colour-vision deficiency [39]. Guideline H reaches the same conclusion from the communication side.

## 6 Implications: design requirements for Tensorgami

The survey translates into ten requirements. They define the drawing style baseline (step 2) and the architecture of the later phases.

**R1 Multiple levels of detail from one model**: L0 system, L1 stages, L2 layers, L3 operators and L4 cell internals (Figure 3), with deterministic collapse and expand.
**R2 Both node semantics**: an operator-centric block view (S4/S5) and a tensor-centric view (S2/S3/S8), rendered from the same intermediate representation.
**R3 Semantic, consistent encoding**: fill colour only for the operation category; shape for the kind of element (data, operation, merge, stochastic, group); one arrow style for data flow, with distinct styles reserved for recurrent state, sampling and training-only signals (guidelines B, H).
**R4 Tensor shapes as first-class annotations** (C×H×W on edges, batch omitted) and optionally to-scale (log) glyph sizes in the volume view.
**R5 Aggregation of repeated structures** into "N×" groups, plus folding of trivial operations (activation into the previous layer, reshape, dropout).
**R6 Automatic legend** listing only the categories that appear.
**R7 Time-aware layouts**: folded and unrolled recurrence and a time × depth grid for RNN, seq2seq and ConvLSTM.
**R8 Explicit stochastic and training-time elements**: sampling nodes, external noise inputs, and dashed loss and gradient paths (VAE, GAN, diffusion).
**R9 Two input routes into one IR**: automatic extraction (PyTorch first, then ONNX and Keras) and a hand-written spec for models that cannot be imported.
**R10 Deterministic vector output** (SVG first, then PDF/PNG) with themeable styles and no heavyweight system dependencies (no Graphviz or LaTeX), so the tool installs locally with `pip`/`uv` alone.

## 7 Conclusion

Drawing deep-learning models is a mature practice without a standard. Nine static styles coexist, each tied to the architecture families and audiences it serves best. The evidence that does exist (the Net2Vis corpus, the TensorBoard

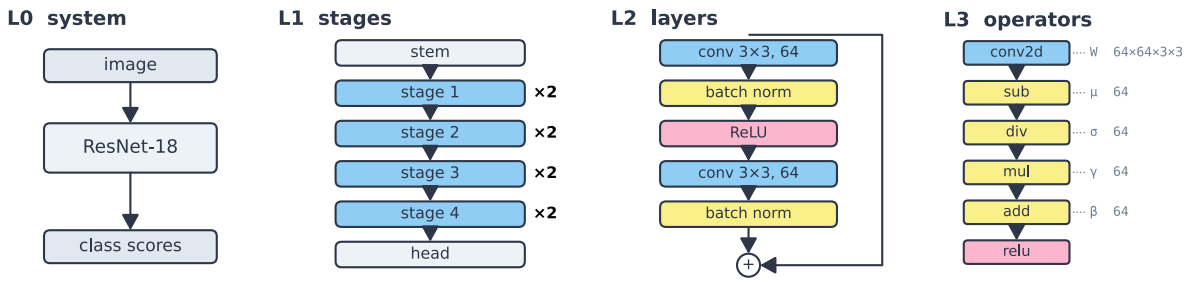



Figure 3: The same residual network at four levels of detail. L4 (cell internals, Figure 1 f) completes the hierarchy for recurrent cells. Tensorgami should produce any of these views from one model description (R1).

design study and Marshall et al.'s reader studies) agrees on a short list of principles: left-to-right flow, semantic rather than decorative colour, a single arrow type for data, aggregation of repeated structure, explicit tensor shapes, automatic legends and several levels of detail. No current tool combines automatic extraction, publication-quality styling and coverage of recurrent, convolutional-recurrent and generative families. Tensorgami's baseline style (step 2) encodes these principles directly.

## References


[1] G. C. Marshall, A. Freitas, and C. Jay, "A Framework for Improving Scholarly Neural Network Diagrams," *arXiv preprint arXiv:2008.12566*, 2020.

[2] G. C. Marshall, A. Freitas, and C. Jay, "An Evidence-Based Guidance Framework for Neural Network System Diagrams," *PLOS ONE*, vol. 20, no. 3, p. e0318800, 2025, doi: 10.1371/journal.pone.0318800.

[3] F. Hohman, M. Kahng, R. Pienta, and D. H. Chau, "Visual Analytics in Deep Learning: An Interrogative Survey for the Next Frontiers," *IEEE Transactions on Visualization and Computer Graphics*, vol. 25, no. 8, 2019, doi: 10.1109/TVCG.2018.2843369.

[4] A. Bäuerle, C. van Onzenoodt, and T. Ropinski, "Net2Vis – A Visual Grammar for Automatically Generating Publication-Tailored CNN Architecture Visualizations," *IEEE Transactions on Visualization and Computer Graphics*, vol. 27, no. 6, 2021, doi: 10.1109/TVCG.2021.3057483.

[5] F. van Veen, "The Neural Network Zoo." [Online]. Available: https://www.asimovinstitute.org/neural-network-zoo/

[6] A. LeNail, "NN-SVG: Publication-Ready Neural Network Architecture Schematics," *Journal of Open Source Software*, vol. 4, no. 33, p. 747, 2019, doi: 10.21105/joss.00747.

[7] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[8] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Advances in Neural Information Processing Systems 25 (NIPS)*, 2012.

[9] H. Iqbal, "PlotNeuralNet v1.0.0." [Online]. Available: https://github.com/HarisIqbal88/PlotNeuralNet

[10] P. Gavrikov, "visualkeras: Architecture Visualization of Keras Models." [Online]. Available: https://github.com/paulgavrikov/visualkeras

[11] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[12] C. Szegedy *et al.*, "Going Deeper with Convolutions," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.

[13] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems 30 (NIPS)*, 2017.

[14] Keras team, "Model plotting utilities (keras.utils.plot_model)." [Online]. Available: https://keras.io/api/utils/model_plotting_utils/

[15] M. Kurttutan, "torchview: Visualize PyTorch Models." [Online]. Available: https://github.com/mert-kurttutan/torchview

[16] E. R. Gansner and S. C. North, "An Open Graph Visualization System and its Applications to Software Engineering," *Software: Practice and Experience*, vol. 30, no. 11, pp. 1203–1233, 2000.

[17] K. Wongsuphasawat *et al.*, "Visualizing Dataflow Graphs of Deep Learning Models in TensorFlow," *IEEE Transactions on Visualization and Computer Graphics (Proc. VAST)*, vol. 24, no. 1, pp. 1–12, 2018.

[18] L. Roeder, "Netron: Visualizer for Neural Network, Deep Learning and Machine Learning Models." [Online]. Available: https://github.com/lutzroeder/netron

[19] S. Zagoruyko, "PyTorchViz: Visualize PyTorch Execution Graphs." [Online]. Available: https://github.com/szagoruyko/pytorchviz

[20] J. Taylor and N. Kriegeskorte, "Extracting and Visualizing Hidden Activations and Computational Graphs of PyTorch Models with TorchLens," *Scientific Reports*, vol. 13, 2023, doi: 10.1038/s41598-023-40807-0.

[21] Google, "Model Explorer: Graph Visualization for Large Model Development." [Online]. Available: https://research.google/blog/model-explorer/

[22] C. Olah, "Understanding LSTM Networks." [Online]. Available: https://colah.github.io/posts/2015-08-Understanding-LSTMs/

[23] K. Cho *et al.*, "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014.

[24] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[25] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to Sequence Learning with Neural Networks," in *Advances in Neural Information Processing Systems 27 (NIPS)*, 2014.

[26] D. Bahdanau, K. Cho, and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," in *International Conference on Learning Representations (ICLR)*, 2015.

[27] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *International Conference on Learning Representations (ICLR)*, 2014.

[28] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015.

[29] V. Abbott, "Neural Circuit Diagrams: Robust Diagrams for the Communication, Implementation, and Analysis of Deep Learning Architectures," *arXiv preprint arXiv:2402.05424*, 2024.

[30] Z. J. Wang *et al.*, "CNN Explainer: Learning Convolutional Neural Networks with Interactive Visualization," *IEEE Transactions on Visualization and Computer Graphics*, vol. 27, no. 2, 2021.

[31] B. Bycroft, "LLM Visualization." [Online]. Available: https://bbycroft.net/llm

[32] D. D. Johnson, "Penzai + Treescope: A Toolkit for Interpreting, Visualizing, and Editing Models as Data," *arXiv preprint arXiv:2408.00211*, 2024.

[33] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.

[34] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[35] X. Shi, Z. Chen, H. Wang, D.-Y. Yeung, W.-kin Wong, and W.-chun Woo, "Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting," in *Advances in Neural Information Processing Systems 28 (NIPS)*, 2015, pp. 802–810.

[36] I. Goodfellow *et al.*, "Generative Adversarial Nets," in *Advances in Neural Information Processing Systems 27 (NIPS)*, 2014.

[37] W. Abdulla, "HiddenLayer: Neural Network Graphs and Training Metrics for PyTorch, TensorFlow and Keras." [Online]. Available: https://github.com/waleedka/hiddenlayer

[38] K. Sugiyama, S. Tagawa, and M. Toda, "Methods for Visual Understanding of Hierarchical System Structures," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 11, no. 2, pp. 109–125, 1981.

[39] M. Okabe and K. Ito, "Color Universal Design (CUD): How to Make Figures and Presentations that are Friendly to Colorblind People." [Online]. Available: https://jfly.uni-koeln.de/color/